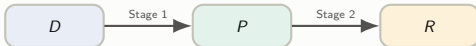


Distribution-Aware Unionable Table Search

Pipeline schematic, short-circuits, and results on santosLarge and WDC (scalability), and the fairified benchmarks santos3, tusSmall3, tusLarge3 (quality)

DUTS experiment log 2026-08-12



Agenda

1. Pipeline schematic

- ▶ $D \rightarrow P \rightarrow R$ and the objectives at each stage
- ▶ Every short-circuit: where and why

2. Fairified benchmarks – quality

- ▶ santos3, tusSmall3, tusLarge3: one fair copy per (query, table) pair
- ▶ Feasibility, precision/recall, ideal recall, ΣU
- ▶ Which stages are bypassed; how often the LP pre-check is decisive
- ▶ Starmie baselines: pure Starmie, exhaustive swap, nested-loop swap

3. santosLarge – scalability

- ▶ Why santosLarge, and choosing attributes/values to maximize $|D|$
- ▶ Choosing $F^*/\delta/\tau$ to maximize reach
- ▶ The first configuration where α actually bites
- ▶ Index cost, per-stage timing at scale

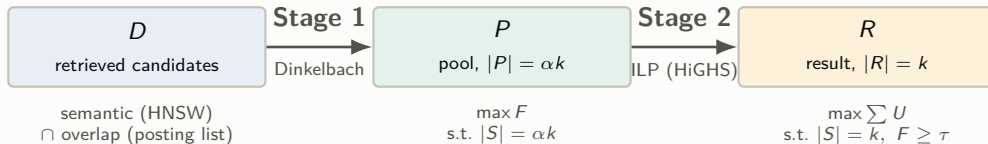
4. WDC – 50.8M-table scalability

- ▶ Sampled tiers; maxD-style query selection replaces the old union-mode relaxation
- ▶ Choosing the $|D|$ threshold, a much larger (k, α) grid, and $F^*/\delta/\tau$
- ▶ Final cohorts: 28 queries (tier_10k), 30 queries (tier_100k); tier_1m pending

5. Takeaways

The two-stage pipeline

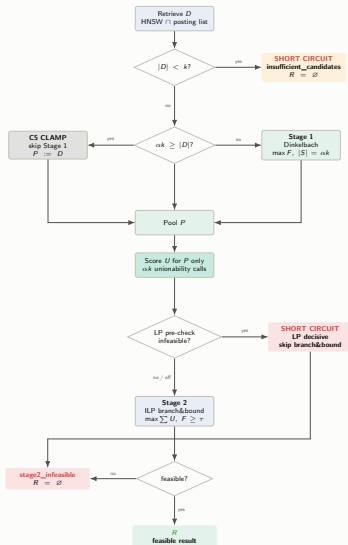
$D \rightarrow P \rightarrow R$, and what each stage optimizes



$R \subseteq P \subseteq D$ always • unionability U computed only for P : exactly αk calls, never $|D|$ or $|T|$
 $F \geq \tau$ is a **floor, not a target** – no upper bound, so results *above* F^* are fully valid, by design

The full process – with every short-circuit

Where an instance can exit early, and why



Short-circuits – what each one means

Short-circuit	Condition	Cost avoided
insufficient_candidates	$ D < k$ before Stage 1 even starts	Stage 1, scoring, Stage 2 – all of it
C5 clamp	$\alpha k \geq D $, so $P := D$	Dinkelbach never runs (<i>not</i> run-and-discarded)
LP pre-check decisive	LP relaxation already infeasible	ILP branch-and-bound
stage2_infeasible	ILP proves no k -subset of P reaches τ	– (this is the answer)

The C4 feasibility certificate (a would-be short-circuit *before* Stage 1, splitting `insufficient_candidates` from an "intrinsically infeasible" cause) was removed from this pipeline entirely – `stage2_infeasible` is now the only post-retrieval failure label.

Fairified benchmarks: setup and what is reported

same pipeline, same metrics on all three – $F^*=0.4$, $\delta=0.1$ ($\tau=0.3$), $top_n=1000$

	santos3	tusSmall3	tusLarge3
fair queries	48	92	142
datalake tables (originals + fair copies)	999	9,178	18,309
mean ground truth tables / query	28.1	852.7	626.7
own fair copies / query (min / mean)	1 / 11.0	16 / 85.9	27 / 100.0
mean $ D $	25.5	122.5	90.5

- ▶ Grid: $k \in \{5, 10, 20\} \times \alpha \in \{10, 5\}$; real pinned-match U ($\sigma=0.6$).
- ▶ **Precision, recall, ΣU and F_R are means over all queries; a failed query (empty result) counts 0. Ideal recall** is over all queries and is identical for every method.
- ▶ Also reported: feasible queries, **bypassed stages** and LP pre-check decisions.

All three are built the same way: one fair copy per (query, table) pair, ratio ≥ 0.4 for the query's value, matched by column name. A table that cannot be augmented for a query is removed from *that query's* ground truth (and from the datalake if it fails for every query), so every ground-truth table has a fair version for its query. The original table and each fair copy are separate unionable tables (both count as hits in precision/recall; the original may not carry enough of the value to reach τ). Ground truth includes every copy of a unionable table. Protected value matched with numeric spelling variants (4/4.0). Details: `fair3-report.md`.

Fairified benchmarks: Quality of the fair result R

$\text{top_n}=1000$, $F^*=0.4$, $\delta=0.1$, $\alpha \in \{10, 5\} \times k \in \{5, 10, 20\}$

bench	α	k	feasible	precision@R	recall@R	ideal recall	mean ΣU	mean F_R
santos3	10	5	42/48	0.875	0.1769	0.2984	52.8	0.388
santos3	10	10	39/48	0.810	0.3005	0.4736	96.0	0.355
santos3	10	20	26/48	0.520	0.3352	0.7299	111.7	0.239
santos3	5	5	42/48	0.846	0.1723	0.2984	51.2	0.405
santos3	5	10	39/48	0.810	0.3005	0.4736	95.9	0.360
santos3	5	20	26/48	0.520	0.3352	0.7299	111.7	0.239
tusSmall3	10	5	89/92	0.933	0.0100	0.0105	46.3	0.364
tusSmall3	10	10	84/92	0.890	0.0197	0.0211	85.6	0.297
tusSmall3	10	20	65/92	0.678	0.0347	0.0421	114.1	0.221
tusSmall3	5	5	89/92	0.920	0.0098	0.0105	46.0	0.393
tusSmall3	5	10	84/92	0.864	0.0192	0.0211	85.1	0.338
tusSmall3	5	20	65/92	0.665	0.0339	0.0421	113.6	0.232
tusLarge3	10	5	133/142	0.790	0.0101	0.0125	50.3	0.348
tusLarge3	10	10	122/142	0.715	0.0190	0.0249	88.9	0.288
tusLarge3	10	20	93/142	0.526	0.0291	0.0498	121.5	0.208
tusLarge3	5	5	133/142	0.782	0.0099	0.0125	49.8	0.375
tusLarge3	5	10	122/142	0.704	0.0184	0.0249	88.2	0.308
tusLarge3	5	20	93/142	0.518	0.0283	0.0498	120.5	0.218

What the quality table shows

findings in plain words – $\alpha=10$, $top_n=1000$

- ▶ **Most queries get a fair answer.** Feasible queries ($k=5/10/20$): santos3 42/39/26 of 48, tusSmall3 89/84/65 of 92, tusLarge3 133/122/93 of 142. Feasible means k tables whose union with the query has at least 30% rows with the protected value.
- ▶ **Precision and recall are means over all queries; a failed query counts 0.** precision@R: santos3 0.88/0.81/0.52, tusSmall3 0.93/0.89/0.68, tusLarge3 0.79/0.72/0.53. They fall with k mainly because more queries fail.
- ▶ **Recall stays below the ideal recall** (the best possible with k tables, the same for every method): santos3 0.18/0.30/0.34 vs 0.30/0.47/0.73, tusSmall3 0.010/0.020/0.035 vs 0.011/0.021/0.042, tusLarge3 0.010/0.019/0.029 vs 0.012/0.025/0.050. The gap is widest where many queries fail (santos3, $k=20$: 26 of 48 feasible).
- ▶ **The fairness target is met** where an answer exists: among the feasible queries $F_R \geq 0.31$ ($\tau=0.30$), about 0.44 on santos3 and 0.31–0.41 on TUS. In the table ΣU and F_R are means over **all** queries with a failed query counting 0, so they are lower where queries fail.

Example: tusSmall3, $\alpha=10$, $k=5$: 89 of 92 feasible, precision 0.933, recall 0.0100 vs ideal 0.0105.

Fairified benchmarks: Stage bypasses, LP pre-check and feasibility

top_n= 1000, $F^*=0.4$, $\delta=0.1$, $\alpha \in \{10, 5\} \times k \in \{5, 10, 20\}$

bench	α	k	$ D $	$ D < k$	S1 skip	S1 ran	LP dec.	MILP ran	feasible	$\geq k$ own	feas. among
santos3	10	5	25.5	4	40	4	2	42	42/48	39	39/39
santos3	10	10	25.5	7	41	0	2	39	39/48	36	35/36
santos3	10	20	25.5	20	28	0	2	26	26/48	3	3/3
santos3	5	5	25.5	4	27	17	2	42	42/48	39	39/39
santos3	5	10	25.5	7	37	4	2	39	39/48	36	35/36
santos3	5	20	25.5	20	28	0	2	26	26/48	3	3/3
tusSmall3	10	5	122.5	0	11	81	3	89	89/92	92	89/92
tusSmall3	10	10	122.5	1	42	49	7	84	84/92	92	84/92
tusSmall3	10	20	122.5	2	77	13	25	65	65/92	85	58/85
tusSmall3	5	5	122.5	0	3	89	3	89	89/92	92	89/92
tusSmall3	5	10	122.5	1	10	81	7	84	84/92	92	84/92
tusSmall3	5	20	122.5	2	41	49	25	65	65/92	85	58/85
tusLarge3	10	5	90.5	6	34	102	3	133	133/142	142	133/142
tusLarge3	10	10	90.5	10	76	56	10	122	122/142	142	122/142
tusLarge3	10	20	90.5	15	119	8	34	93	93/142	142	93/142
tusLarge3	5	5	90.5	6	10	126	3	133	133/142	142	133/142
tusLarge3	5	10	90.5	10	30	102	10	122	122/142	142	122/142
tusLarge3	5	20	90.5	15	71	56	34	93	93/142	142	93/142

What the stage table shows

how each query ends, and which stages were skipped – $\alpha=10$, $top_n=1000$

- ▶ Every query ends one of three ways: $|D| < k$ (too few candidates – Stage 1 and 2 skipped), **LP decisive** (the LP proves no k tables reach $F \geq 0.3$ – MILP skipped) or **MILP ran** (feasible; the MILP never failed after a feasible LP).
- ▶ **santos3**: the failures are mostly too few candidates: $|D| < k$ for 4 / 7 / 20 of the 6 / 9 / 22 infeasible queries at $k=5/10/20$. Own copies are a sufficient, not a necessary, condition: only 3 queries have ≥ 20 own fair copies, yet 26 are feasible at $k=20$, because the kept original tables and other queries' copies fill the slots. Among queries with $\geq k$ own copies: 39/39, 35/36, 3/3.
- ▶ **TUS**: every query has at least k own copies for $k \leq 10$, yet 3–20 queries per row are infeasible – retrieval did not bring enough of them into D (next slides).
- ▶ **Stage 1 is bypassed** ($P=D$) for most santos3 queries and for most TUS queries at $k=20$ (77/92, 119/142); at $k=5$ it runs for most TUS queries (81/92, 102/142). The LP pre-check is decisive for 2–34 queries per row, each skipping a MILP call.

Columns: $|D|$ mean candidates; $|D| < k$ both stages skipped; S1 skip Stage 1 bypassed ($P=D$); S1 ran; LP dec. LP pre-check decisive (MILP skipped); MILP ran; $\geq k$ own queries with at least k own fair copies; feas. among feasible among those. Example: tusSmall3, $\alpha=10$, $k=20$: $92 = 2 (|D| < k) + 25$ (LP infeasible) + 65 (feasible).

The optimizer cannot fix these failures: it only chooses inside D .

Fairified benchmarks: Precision and recall at D, P and R

$\text{top_n}=1000$, $F^*=0.4$, $\delta=0.1$, $\alpha \in \{10, 5\} \times k \in \{5, 10, 20\}$

bench	α	k	prec@D	rec@D	prec@P	rec@P	prec@R	rec@R
santos3	10	5	0.899	0.8216	0.813	0.7266	0.875	0.1769
santos3	10	10	0.899	0.8216	0.769	0.6910	0.810	0.3005
santos3	10	20	0.899	0.8216	0.516	0.5233	0.520	0.3352
santos3	5	5	0.899	0.8216	0.799	0.6100	0.846	0.1723
santos3	5	10	0.899	0.8216	0.766	0.6724	0.810	0.3005
santos3	5	20	0.899	0.8216	0.516	0.5233	0.520	0.3352
tusSmall13	10	5	0.934	0.2585	0.923	0.0953	0.933	0.0100
tusSmall13	10	10	0.934	0.2585	0.916	0.1729	0.890	0.0197
tusSmall13	10	20	0.934	0.2585	0.912	0.2352	0.678	0.0347
tusSmall13	5	5	0.934	0.2585	0.928	0.0487	0.920	0.0098
tusSmall13	5	10	0.934	0.2585	0.913	0.0952	0.864	0.0192
tusSmall13	5	20	0.934	0.2585	0.905	0.1728	0.665	0.0339
tusLarge3	10	5	0.746	0.1570	0.723	0.0767	0.790	0.0101
tusLarge3	10	10	0.746	0.1570	0.694	0.1226	0.715	0.0190
tusLarge3	10	20	0.746	0.1570	0.657	0.1456	0.526	0.0291
tusLarge3	5	5	0.746	0.1570	0.744	0.0444	0.782	0.0099
tusLarge3	5	10	0.746	0.1570	0.696	0.0764	0.704	0.0184
tusLarge3	5	20	0.746	0.1570	0.659	0.1211	0.518	0.0283

What the precision/recall table shows

the same measures at D , P and R , all queries – $\alpha=10$, $top_n=1000$

- ▶ D : tables retrieved (before any selection). P : the pool of αk tables chosen by Stage 1. R : the final k tables. An empty or missing set (no pool because $|D| < k$, or no result because the query failed) counts 0.
- ▶ **The retrieved set is mostly unionable:** precision@D 0.90 (santos3), 0.93 (tusSmall3), 0.75 (tusLarge3); recall@D 0.82/0.26/0.16.
- ▶ **Precision at P and R stays close to D where queries succeed**, and drops at large k because queries fail: santos3 precision@R 0.88/0.81/0.52 (only 26 of 48 queries have a result at $k=20$).
- ▶ **Recall falls from D to R because the sets shrink** (D holds $\sim 25 / 120 / 90$ tables, R holds k) and because failed queries count 0.

Example: tusSmall3, $\alpha=10$, $k=5$: precision $0.934 \rightarrow 0.923 \rightarrow 0.933$ and recall $0.259 \rightarrow 0.095 \rightarrow 0.0100$ at $D \rightarrow P \rightarrow R$. Precision/recall at D do not depend on k or α .

Why some queries stay infeasible: own fair copies in D

$top_n=1000$ – each query has its own fair copies (ratio ≥ 0.4 for its value) in the datalake

benchmark	mean $ D $	own copies in D	own copies in datalake	ratio over D	ratio, own copies in D
santos3	25.5	9.9	11.0	0.372	0.485
tusSmall3	122.4	17.1	85.9	0.152	0.396
tusLarge3	83.8	13.6	100.0	0.173	0.386

- ▶ **santos3:** retrieval brings almost all of the query's own copies into D (9.9 of 11.0) and the ratio over D (0.37) is above $\tau=0.3$; the failures come mostly from queries with fewer than k candidates.
- ▶ **TUS:** only ~ 17 of ~ 86 (tusSmall3) and ~ 14 of 100 (tusLarge3) own copies reach D ; the ratio over all of D (0.15–0.17) is below τ . Copies of one table have nearly identical vectors, so the top_n window fills with originals and with copies made for *other* queries, where the query's value is rare. 9 of 142 tusLarge3 queries get none.
- ▶ Retrieval is not fairness-aware and the optimizer only sees D : these failures are a retrieval limit, and the MILP never failed.

Separate run of the same code (own HNSW build), so $|D|$ differs slightly from the tables. Details: `fair3-report.md`.

Starmie baselines: what was run

same fair queries, datalake and ground truth as the DUTS tables – no timings reported

- ▶ **Pure Starmie:** the N nearest columns of every query column give the candidates; each is scored by bipartite column matching; the top- k is returned. No protected attribute is used.
- ▶ **Exhaustive swap (no filtering):** candidates are scored with the protected column *pinned* in the matching; if the initial top- k has $F < \tau$, violating tables are swapped out for the best replacement, trying every candidate.
- ▶ **Nested-loop swap (winnow filtering):** the same swap, but the candidates are first pruned by a dominance filter (a candidate is dropped if another has more protected rows, fewer other rows and at least as good unionability).
- ▶ Fixed for all three: $N=1000$, $\sigma=0.6$, $F^*=0.4$, $\delta=0.1$ ($\tau=0.3$), $k \in \{5, 10, 20\}$, on santos3 (48 queries), tusSmall3 (92), tusLarge3 (142).
- ▶ **Metrics, recomputed by our code as for DUTS.** Precision, recall and ideal recall are means over all queries. For the two swap methods an infeasible query (not exactly k tables with $F \geq \tau$) counts 0 for precision, recall, ΣU and F_R . **Pure Starmie is scored on what it returns, regardless of τ** , with the number of queries that reached τ shown beside it; its precision, recall, ΣU and F are over all queries.

The baselines are Starmie's own code (HNSWSearcher_Fair), run unmodified except for the three changes listed on the next slide. Because pure Starmie is not zeroed when it misses τ , its precision and recall count unfair lists and are not directly comparable with the swap methods'.

Starmie baselines: defaults and changes

taken from the Starmie code, not chosen by us

- ▶ **Index:** HNSW, cosine, $M=32$, $ef_{\text{construction}}=100$, $ef=10$ (a query with $N=1000$ uses $\max(ef, N)$); tables shuffled with seed 42 at scale 1.0.
- ▶ **Matching:** an edge counts if cosine $> \sigma$. We use $\sigma=0.6$, as DUTS; Starmie's own default for these benchmarks is 0.7.
- ▶ **Dominance rules** of the nested-loop filter: the bundled `preference_config.json` (protected count, non-protected count, unionability score), unchanged.
- ▶ **Swap algorithms:** default removal and replacement order; no parameters were set.
- ▶ **F:** Starmie's `compute_F` over the benchmark's `metadata_combined.pkl`: any column counts as categorical (no $\theta_{\text{cat}}=50$ limit); a table adds rows only if its protected column aligns in the constrained matching; the query is included. Pure Starmie is scored after the fact with the same alignment.
- ▶ **Our three changes:** (1) the protected value is matched with its numeric spelling variants (4/4.0), as in the DUTS runs; (2) the raw datalake is not loaded into memory and the metadata pickle is not rewritten; (3) the index is written to a scratch folder.
- ▶ **Metrics:** recomputed from the returned lists (Starmie's own script averages precision only over queries with $|gt| \geq k$ and uses an uncapped ideal recall, so it is not comparable).
- ▶ **ΣU :** Starmie's matching score of the returned tables; it equals the DUTS pinned-match score in all but 20 of 2,538 rows.

Pure Starmie (no fairification): results

$N=1000$, $\sigma=0.6$, $F^*=0.4$, $\delta=0.1$

bench	k	reached τ	precision	recall	ideal recall	mean ΣU	mean F
santos3	5	21/48	0.888	0.2772	0.2984	60.0	0.337
santos3	10	12/48	0.829	0.4216	0.4736	115.7	0.269
santos3	20	9/48	0.752	0.6516	0.7299	220.0	0.206
tusSmall3	5	7/92	0.900	0.0098	0.0105	50.2	0.161
tusSmall3	10	5/92	0.886	0.0195	0.0211	100.1	0.126
tusSmall3	20	1/92	0.843	0.0358	0.0421	198.5	0.094
tusLarge3	5	22/142	0.792	0.0099	0.0125	55.9	0.183
tusLarge3	10	13/142	0.769	0.0191	0.0249	111.0	0.143
tusLarge3	20	8/142	0.713	0.0348	0.0498	219.4	0.108

Pure Starmie is scored on the list it returns, *regardless of* τ : precision, recall, ideal recall, ΣU and F are means over **all** queries. *reached τ* : queries whose unfair top- k has $F \geq \tau=0.3$.

What the pure Starmie table shows

findings in plain words – all queries, as returned, regardless of τ

- ▶ **The unfair top- k rarely meets the fairness target.** It has $F \geq 0.3$ for only santos3 21/12/9 of 48, tusSmall3 7/5/1 of 92, tusLarge3 22/13/8 of 142 queries ($k=5/10/20$).
- ▶ **It gets worse as k grows.** Mean F over all queries: santos3 0.34/0.27/0.21, tusSmall3 0.16/0.13/0.09, tusLarge3 0.18/0.14/0.11; ΣU grows with k (60/116/220 on santos3).
- ▶ **Precision over all queries:** 0.89/0.83/0.75 (santos3), 0.90/0.89/0.84 (tusSmall3), 0.79/0.77/0.71 (tusLarge3).
- ▶ **Recall vs ideal recall:** santos3 0.28/0.42/0.65 vs 0.30/0.47/0.73; tusSmall3 0.010/0.019/0.036 vs 0.011/0.021/0.042; tusLarge3 0.010/0.019/0.035 vs 0.012/0.025/0.050.

These are the accuracy of Starmie's own (mostly unfair) lists; the swap methods are zeroed when they fail, so their numbers are lower by construction.

Pure Starmie (no fairification): algorithm details

algorithm-specific measures, $N=1000$, $\sigma=0.6$

bench	k	cand.	$< k$ ret.
santos3	5	105.8	0
santos3	10	105.8	0
santos3	20	105.8	0
tusSmall3	5	144.4	0
tusSmall3	10	144.4	0
tusSmall3	20	144.4	0
tusLarge3	5	126.5	0
tusLarge3	10	126.5	0
tusLarge3	20	126.5	0

cand.: mean tables verified after the $N=1000$ nearest-column lookup, before taking the top- k . $< k$ *ret.*: queries with fewer than k tables returned. No swap is done, so there is no swap information; the reached- τ count and F are in the results table.

Exhaustive swap (no filtering): results

$N=1000$, $\sigma=0.6$, $F^*=0.4$, $\delta=0.1$

bench	k	feasible	precision	recall	ideal recall	mean ΣU	mean F_R
santos3	5	47/48	0.875	0.2502	0.2984	55.7	0.397
santos3	10	42/48	0.704	0.3113	0.4736	93.9	0.338
santos3	20	31/48	0.441	0.3340	0.7299	106.4	0.242
tusSmall3	5	66/92	0.676	0.0085	0.0105	29.6	0.239
tusSmall3	10	43/92	0.429	0.0135	0.0211	32.8	0.151
tusSmall3	20	24/92	0.227	0.0202	0.0421	29.4	0.081
tusLarge3	5	108/142	0.628	0.0086	0.0125	37.4	0.264
tusLarge3	10	77/142	0.399	0.0129	0.0249	44.8	0.181
tusLarge3	20	49/142	0.205	0.0166	0.0498	44.7	0.111

precision, recall, ideal recall: means over **all** queries; a query that is not feasible (exactly k tables with $F \geq \tau=0.3$) counts 0 for precision, recall, ΣU and F_R . When the exhaustive swap cannot reach τ it returns nothing.

What the exhaustive-swap table shows

findings in plain words – $N=1000$, $\sigma=0.6$

- ▶ **Swapping makes many more queries feasible** than pure Starmie reaches: santos3 47/42/31 of 48, tusSmall3 66/43/24 of 92, tusLarge3 108/77/49 of 142 – but the number drops quickly as k grows.
- ▶ **Precision and recall over all queries (failed = 0)** fall with k for that reason: precision 0.88/0.70/0.44 (santos3), 0.68/0.43/0.23 (tusSmall3), 0.63/0.40/0.21 (tusLarge3); recall 0.25/0.31/0.33, 0.0085/0.0135/0.0202, 0.0086/0.0129/0.0166 against ideal recall 0.30/0.47/0.73, 0.011/0.021/0.042, 0.012/0.025/0.050.
- ▶ **Almost every query needs the swap:** the initial top- k violates the target for 28/36/39 of 48 (santos3), 85/87/90 of 92 (tusSmall3), 122/129/134 of 142 (tusLarge3); it succeeds for, e.g., 27/28, 30/36, 22/39 on santos3.
- ▶ **When it fails it returns nothing** (1/6/17 empty results on santos3, 26/49/68 on tusSmall3, 34/65/93 on tusLarge3) and **it stops as soon as the target is met:** among the feasible queries mean F_R is only just above 0.3 (0.31–0.41); the table's ΣU and F_R include the failed queries as 0.

Exhaustive swap (no filtering): algorithm details

algorithm-specific measures, $N=1000$, $\sigma=0.6$

bench	k	cand.	$< k$ ret.	needed	succeeded	swaps	F before
santos3	5	106.2	1	28/48	27/28	1.6	0.221
santos3	10	106.2	6	36/48	30/36	3.4	0.170
santos3	20	106.2	17	39/48	22/39	5.3	0.131
tusSmall3	5	144.9	26	85/92	59/85	2.4	0.144
tusSmall3	10	144.9	49	87/92	38/87	3.6	0.115
tusSmall3	20	144.9	68	90/92	22/90	3.7	0.089
tusLarge3	5	125.5	34	122/142	88/122	2.2	0.150
tusLarge3	10	125.5	65	129/142	64/129	3.4	0.120
tusLarge3	20	125.5	93	134/142	41/134	3.6	0.093

cand.: mean candidates scored. *< k ret.*: queries with fewer than k tables returned (swap failed, empty list). *needed*: the initial top- k had $F < \tau$, so swaps ran. *succeeded*: swaps brought F to $\geq \tau$ (out of those that needed it). *swaps*: mean swaps made, over the queries that needed fairification (failed ones included). *F before*: mean F of the initial top- k of those queries.

Nested-loop swap (winnow filtering): results

$N=1000$, $\sigma=0.6$, $F^*=0.4$, $\delta=0.1$

bench	k	feasible	precision	recall	ideal recall	mean ΣU	mean F_R
santos3	5	47/48	0.846	0.2277	0.2984	55.2	0.392
santos3	10	35/48	0.612	0.2495	0.4736	83.3	0.292
santos3	20	20/48	0.317	0.2348	0.7299	68.9	0.170
tusSmall3	5	52/92	0.528	0.0076	0.0105	20.1	0.189
tusSmall3	10	33/92	0.320	0.0116	0.0211	22.3	0.114
tusSmall3	20	12/92	0.105	0.0095	0.0421	13.7	0.040
tusLarge3	5	101/142	0.569	0.0083	0.0125	32.1	0.245
tusLarge3	10	59/142	0.284	0.0109	0.0249	28.7	0.141
tusLarge3	20	39/142	0.150	0.0134	0.0498	32.2	0.090

precision, recall, ideal recall: means over **all** queries; a query that is not feasible (exactly k tables with $F \geq \tau=0.3$) counts 0 for precision, recall, ΣU and F_R . The nested-loop swap returns k tables even when it failed; that unfair list is discarded (counts 0).

What the nested-loop-swap table shows

findings in plain words – $N=1000$, $\sigma=0.6$

- ▶ **Fewer feasible queries than the exhaustive swap** in every row except santos3 at $k=5$: santos3 47/35/20 of 48, tusSmall3 52/33/12 of 92, tusLarge3 101/59/39 of 142.
- ▶ **Precision and recall over all queries (failed = 0)** are correspondingly lower: precision 0.85/0.61/0.32 (santos3), 0.53/0.32/0.11 (tusSmall3), 0.57/0.28/0.15 (tusLarge3); recall 0.23/0.25/0.23, 0.0076/0.0116/0.0095, 0.0083/0.0109/0.0134.
- ▶ **The winnow filter removes 89–96% of the swap candidates** (e.g. tusSmall3, $k=5$: 10,828 of 11,306), leaving on average only 2–6 candidates per query – fewer chances to find a replacement that reaches τ .
- ▶ **It always returns k tables, even when the swap failed**; those unfair lists are discarded here (counted 0, also for ΣU and F_R). Among the feasible queries mean F_R is 0.31–0.41.

Nested-loop swap (winnow filtering): algorithm details

algorithm-specific measures, $N=1000$, $\sigma=0.6$

bench	k	cand.	$< k$ ret.	needed	succeeded	swaps	F before	removed / pool	share	left
santos3	5	106.1	0	28/48	27/28	1.8	0.221	1876/2102	0.892	6.2
santos3	10	106.1	0	36/48	23/36	3.2	0.170	2406/2673	0.900	4.2
santos3	20	106.1	0	39/48	11/39	3.2	0.131	2330/2593	0.899	3.5
tusSmall3	5	144.7	0	85/92	45/85	2.2	0.145	10828/11306	0.958	3.4
tusSmall3	10	144.7	0	87/92	28/87	2.8	0.114	10709/11184	0.958	2.7
tusSmall3	20	144.7	0	90/92	10/90	3.0	0.088	10619/11077	0.959	2.1
tusLarge3	5	125.1	0	121/142	80/121	2.2	0.149	12042/13031	0.924	6.0
tusLarge3	10	125.1	0	128/142	45/128	3.2	0.122	12371/13443	0.920	5.2
tusLarge3	20	125.1	0	134/142	31/134	3.3	0.094	12585/13694	0.919	5.0

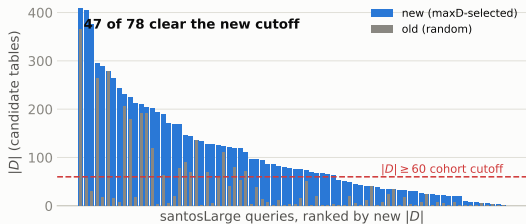
Columns as for the exhaustive swap, plus the winnow filter: *removed / pool*: candidates dropped as dominated out of those considered, summed over queries; *share*: their ratio; *left*: mean candidates left for swapping. The nested-loop swap returns its last (possibly unfair) state on failure, so it never returns fewer than k .

Why santosLarge, and why $|D| \geq 50$

santosLarge (11,086 datalake tables, top_n=5000), with the *original* random attribute/value assignment

- ▶ Median $|D|$ is *still* only 11 – most queries' protected value occurs in very few tables
- ▶ The tail is long but thin: only **19 of 78** queries reach $|D| \geq 50$
- ▶ Restricting to that 19-query tail was the only way to see α bite at all – until now (next slide)

Instead of restricting to a thin 19-query tail, we re-picked *which* attribute/value each query targets – deliberately, not randomly – so far more of the 78 queries carry a usable $|D|$. Methodology and the resulting cohort are the next two slides; every santosLarge slide after them uses that new cohort.



santosLarge: choosing attributes/values to maximize $|D|$

Why random assignment fails, and what replaces it

Why the original (random) assignment gives small $|D|$

- ▶ Each query's protected attribute/value came from starmie_fair's `experimental_setup.py` (`selection_strategy='random'`, seed 42): pick a random categorical column, then `random.choice(unique_values)` – with no regard for how common that value is *elsewhere* in the 11,086-table datalake.
- ▶ $|D|$ is driven entirely by cross-table value popularity (via the overlap posting-list) intersected with semantic column similarity – a purely random value usually lands on something rare.

The replacement: rank by a free proxy, verify by real retrieval

- ▶ Per query table: enumerate every (categorical attr, value) pair from its own synopsis (excluding the empty-cell bucket).
- ▶ Rank all candidates by `|postings[value]|` – the overlap index's own posting-list size – a free, no-retrieval proxy for "how many (*table*, *attr*) pairs would pass the overlap filter."
- ▶ Verify the top 20 candidates by *actual* retrieval (`top_n=5000`, same context every santosLarge script uses); keep whichever gives the single largest real $|D|$ – **maximize, no target, no early exit.**

This is a deliberate, documented departure from the original random selection, not a bug fix – see next: it systematically favors common/generic values over rare ones, which is exactly what makes $|D|$ large.

santosLarge: what changed, concretely

Old (random) vs. new (maxD) assignment, three examples

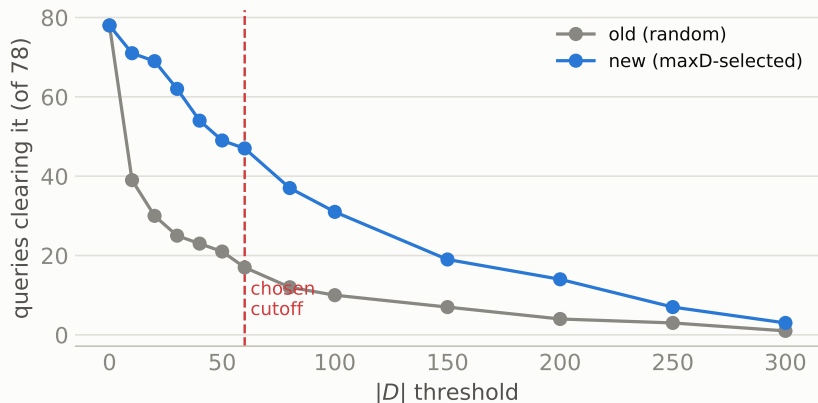
Table	Old: attr $\rightarrow$ value ($ D $)	New: attr $\rightarrow$ value ($ D $)
cabinet-office...	Organisation $\rightarrow$ Cabinet Office (17)	Valid? $\rightarrow$ 1 (213)
1004ipopayments.csv	Date $\rightarrow$ 20/04/2010 (2)	Expense type $\rightarrow$ Other (134)
Dec_19.csv	(unnamed) $\rightarrow$ 09/12/2019 (6)	(unnamed) $\rightarrow$ Entity (171)

The pattern is consistent: new values skew heavily toward generic/structural tokens (0, 1.0, Headcount, Entity, Other) rather than the specific, often-unique values random selection picked (a particular date, org name, or country). That is the mechanism behind the $|D|$ gain – these are values that recur across many tables in this GOV.UK-style organizational-report corpus, not meaningfully rare "protected" attributes anymore.

A handful of tables (~ 4 of 78) improve little or not at all even after checking up to ~ 200 candidates each (e.g. biodiversity_by_county.csv, $|D|=1 \rightarrow 1$) – a genuine intrinsic limit (no other table is both semantically similar and value-overlapping), not a search-budget problem.

santosLarge: picking the $|D|$ threshold and (k, α) grid

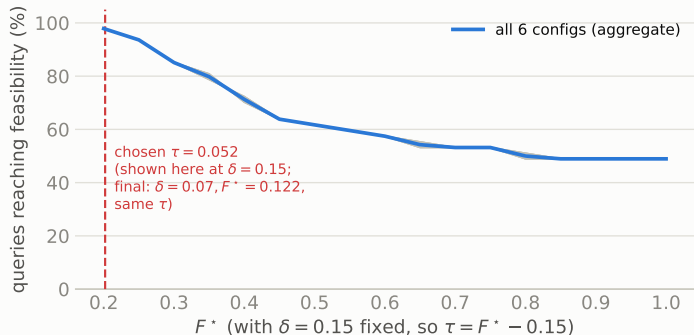
Data-driven, not carried over from the old grid



Old (random) $|D|$: median 11, max 365. New (maxD): median 77, mean 108, max 408. With the old grid ($k \in \{10, 20\}$, $\alpha \in \{2, 3\}$, max pool 60) carried over unexamined, the pool size would say nothing about the new distribution – so the grid is re-derived from it instead: $k \in \{10, 15\}$, $\alpha \in \{2, 3, 4\}$ (pool sizes 20–60) and cohort $|D| \geq 60$, chosen so every retained query stays *unclamped across all six cells* – one consistent, comparable population for the whole sweep, same design principle as the original

santosLarge: choosing F^* , δ , and τ

Reach-maximizing



Only $\tau = F^* - \delta$ enters anything mechanically (U – hence δ/F^* individually – never appears in Stage 2's constraints, only its objective), so “maximize reach” with no floor trivially wants $\tau \rightarrow 0$. We fixed δ , swept F^* from the $\tau \geq 0.05$ floor, and re-solved Stage 2 only across all $47 \times 6 = 282$ cells (candidates scored $U=1.0$ – feasibility doesn't depend on U). **Reach is uniform across α at fixed k** – α creates no bottleneck here. santosLarge is the *scalability* benchmark. $\delta=0.70$ (tried first) made the tolerance band nearly meaningless; tightened to $\delta=0.07$. For τ : rather than sit at the floor, found the largest τ keeping every query feasible in every config – breaks at $\tau=0.0533$ – so used $\tau=0.052$, $F^*=0.122$.

Sweep cohort: the same $n = 47$, $|D| \geq 60$ population as the previous slide. See [experiments/santoslarge tau sweep.py](#).

santosLarge: one outlier, and why we exclude rather than relax

Getting to 100% honestly

- ▶ At $\tau=0.05$: **46 of 47** queries reach feasibility in every one of the 6 (k, α) cells – 98% overall (276/282), matching the previous slide.
- ▶ The entire gap is **one single query**: SpendoverC2A3500Apr17.csv, infeasible in all 6 of its configs at $\tau=0.05$ – but feasible again at $\tau \leq 0.03$. Its achievable F (even after Stage 1's $\arg \max F$) sits just under the floor.
- ▶ **Two ways to reach 100%**: lower the floor to $\tau=0.03$ for everyone (undoes the floor we deliberately set, to rescue one outlier), or **exclude that one query** and keep $\tau=0.05$.

We exclude it (Option 3). Lowering the floor to fix one outlier would mean the floor was never really a floor – and it's a narrower, more honest exclusion than relaxing a constraint for the whole cohort.

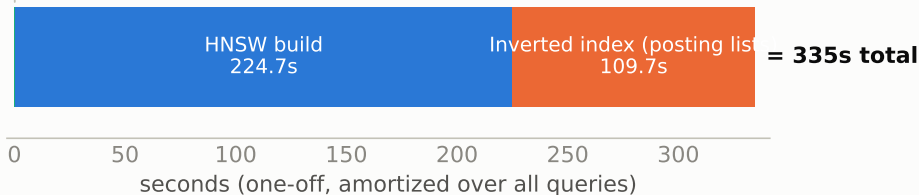
The remaining $n = 46$ queries reach exactly **100%** feasibility (276/276) at

$\tau=0.052$, $F^*=0.122$, $\delta=0.07$ already chosen – not a relaxed target, the one already agreed on.

Final cohort for every santosLarge slide from here on: $n=46$ ($|D| \geq 60$, minus SpendoverC2A3500Apr17.csv) $\times 6$ configs ($k \in \{10, 15\}, \alpha \in \{2, 3, 4\}$) = 276 rows. See `experiments/santoslarge_maxd_study.py`.

santosLarge: one-off index cost

Load vectors, 0.33s

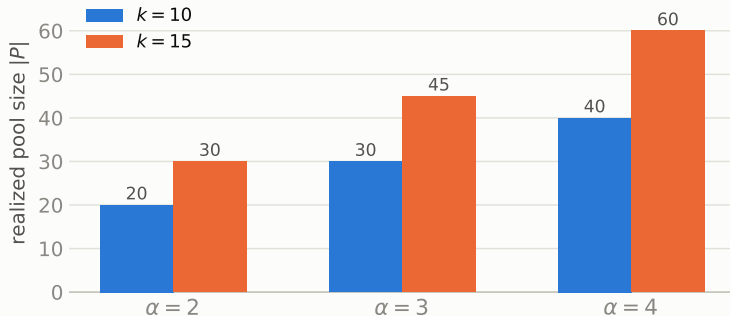


The HNSW (semantic) index now dominates build cost (225–272s across two rebuilds), ahead of the posting-list (overlap) index at a stable 109–110s – both read every one of 11,086 tables. **HNSW build time varies ~21% run-to-run** (the posting-list build does not): `hnswlib`'s multi-threaded `add_items` makes graph construction order, and hence wall-clock cost, non-deterministic even with a fixed seed.

On disk, the HNSW index (**281.8 MB**, 84,157 elements $\times$ 768 dims) is $\sim 17\times$ larger than the posting-list index (**16.9 MB**, 277,704 distinct values). Amortized over all queries; not part of any per-query timing on the following slides.

One-off, over all 78 santosLarge queries – built once, independent of the $n=46$ cohort used below.

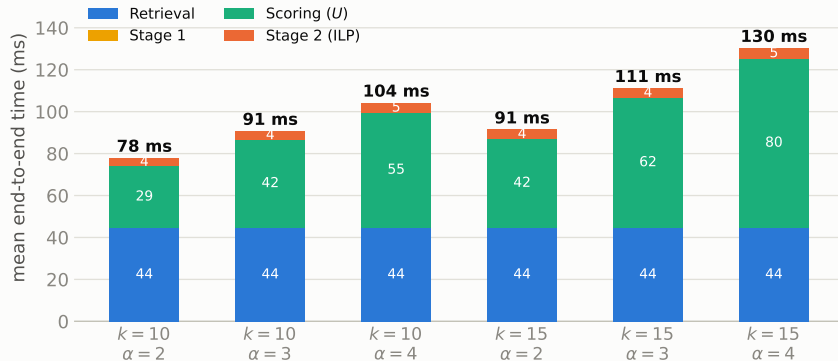
santosLarge: α finally bites



Realized pool size $|P|$ tracks the requested αk **exactly** (20 / 30 / 40 / 30 / 45 / 60) – the C5 clamp fires on **0 of 276** rows here: the $|D| \geq 60$ cohort was chosen specifically so no cell clamps.

Cohort: $n = 46$ per config, all feasible – P is built before Stage 2 runs, so this holds regardless of outcome (here, every row reaches τ anyway – see the outcome slide two ahead).

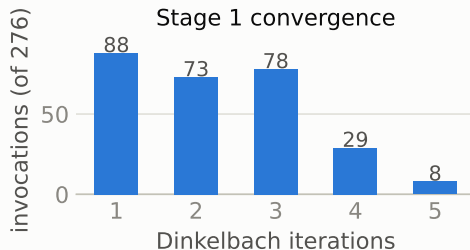
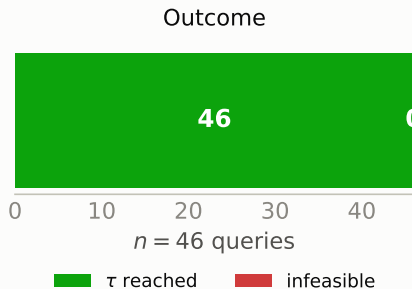
santosLarge: per-stage time scales with αk , not $|D|$



This is the efficiency claim, made visible. Retrieval is flat (fixed cost of probing the index); **scoring grows with αk** (roughly 20 ms at $\alpha k=20$ up to over 100 ms at $\alpha k=60$); Stage 1 stays under a tenth of a millisecond throughout, Stage 2 a few milliseconds.

Cohort: $n = 46$ per config, all feasible – every query is timed the same way regardless of the (now uniformly positive) outcome.

santosLarge: outcome and Stage 1 convergence



All 46 of 46 queries reach τ in every (k, α) cell – by construction of the cohort (Option 3: the one query that didn't was excluded, not papered over with a lower floor). Dinkelbach runs on every row (never clamped) and converges in 1–5 iterations (mean 2.3).

Cohort: left panel, $n = 46$ (one config shown – 46 reach τ , 0 don't, identically across all 6 configs); right panel, $n = 276$ Dinkelbach invocations out of 276 rows (46 queries $\times$ 6 configs) – none are C5-clamped.

santosLarge: convergence time and iterations by pool size

Mean per query, $n = 46$ at each pool size

k	α	pool (αk)	Stage 1 time	mean iters	range
10	2	20	0.317 ms	2.11	1–5
10	3	30	0.370 ms	2.26	1–5
10	4	40	0.391 ms	2.43	1–5
15	2	30	0.368 ms	2.26	1–5
15	3	45	0.383 ms	2.30	1–5
15	4	60	0.388 ms	2.20	1–5

Convergence time tracks pool size only weakly – 0.317 ms at pool 20 to 0.391 ms at pool 40, a $\sim 23\%$ rise for a $2\times$ larger pool – since each iteration is one sort-and-select over the pool, not a re-scan proportional to some larger structure. **Iteration count doesn't increase monotonically with pool size at all** (pool 45's mean 2.30 exceeds pool 60's 2.20): convergence speed is a property of how skewed the pool's N_i/n_i ratios are, not of how large the pool is.

Cohort: $n = 46$ per row, all Dinkelbach invocations (none clamped). See `experiments/RESULTS-santoslarge.md §4` for the full breakdown.

santosLarge: convergence vs. *input* pool size $|D|$

Setting: $k=10$, $\alpha=3$ fixed (output pool $\alpha k=30$); top_n swept from 5,000

requested top_n	actual mean $ D $	clamped	Stage 1 time	mean iters
5,000	160.4	0/46	0.196 ms	2.26
10,000	208.6	0/46	0.249 ms	2.22
15,000	236.0	0/46	0.281 ms	2.26
20,000	250.6	0/46	0.293 ms	2.28
25,000	257.9	0/46	0.302 ms	2.28
30,000	259.0	0/46	0.301 ms	2.28

The previous slide varies the pool Stage 1 *selects* (αk) at fixed top_n; this one fixes k, α and varies the pool Stage 1 *searches over* ($|D|$) by sweeping top_n – top_n is the dial, **realized $|D|$** is what's reported (yield $\neq$ top_n after the overlap filter). Swept from 5,000 since smaller top_n clamps ($\alpha k=30 \geq |D|$, 18/46 at top_n= 500 in an earlier pass), so every point here is clamp-free. **Time scales with $|D|$** – $\sim 1.5\times$ from $|D|\approx 160$ to 259 – one $O(|D| \log |D|)$ sort per iteration. **$|D|$ plateaus** past top_n $\approx 20,000$ (+3% over a 50% larger top_n) as the finite overlap-matching pool is exhausted, and time flattens in step. **Iteration count stays flat** (2.22–2.28): convergence takes only a few iterations regardless of search-pool size – the $|D|$ -dependence is in per-iteration sort cost, not iteration count.

Same 46-query cohort, re-retrieved per top_n. Script: experiments/santoslarge_topn_pool_sweep.py. See experiments/RESULTS-santoslarge.md §4b.

santosLarge: precheck + actual solver only

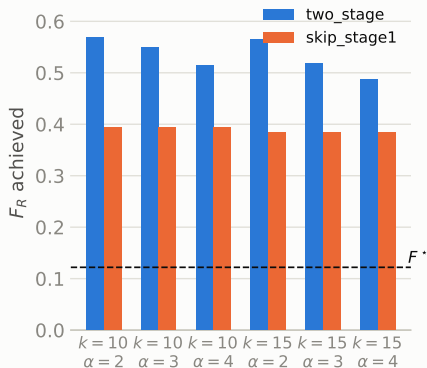
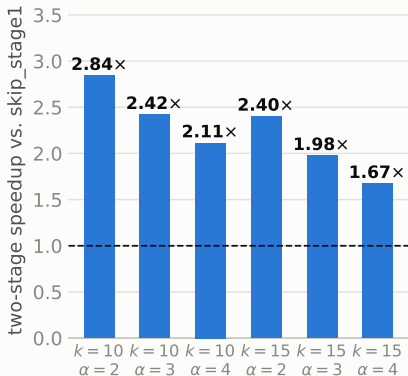
Excludes constraint-setup/verification overhead

α	k	pool	LP precheck	MILP solver	precheck+solver	full Stage 2
2	10	20	0.677 ms	1.164 ms	1.842 ms	2.055 ms
2	15	30	0.677 ms	1.276 ms	1.953 ms	2.160 ms
3	10	30	0.654 ms	1.092 ms	1.745 ms	1.946 ms
3	15	45	0.757 ms	1.356 ms	2.112 ms	2.337 ms
4	10	40	0.679 ms	1.338 ms	2.017 ms	2.224 ms
4	15	60	0.783 ms	1.507 ms	2.291 ms	2.518 ms

"Full Stage 2" (§5) is precheck + **actual MILP solve** + constraint-setup/post-solve-verification overhead ($\sim 0.2\text{--}0.25$ ms, small and roughly constant). Isolating just the two solves: the **MILP solver dominates** ($\sim 60\text{--}65\%$ of precheck+solver time), the LP precheck the remaining $\sim 35\text{--}40\%$, both growing mildly with pool size.

Caveat: absolute ms here differ from §5's LP-time column for identical (k, α) cells – sub-millisecond perf_counter timings are noisy across process runs ($\pm 30\%$ or so); the relative split (solver > precheck) is the stable finding. See RESULTS-santoslarge.md §5c.

santosLarge: two-stage vs. skipping Stage 1



Stage 1 earns its place on **speed** (1.7–2.8 $\times$ faster, growing with αk) but costs ΣU : skip_stage1 optimizes over the superset $D \supseteq P$, so it can only match or beat two-stage there (roughly 5–9% higher ΣU here). Both satisfy $F_R \geq \tau$; F_R 's distance from $F^*=0.122$ is a diagnostic, not a constraint either violates – achieved F_R sits around 0.49–0.57 on average (individual rows range 0.052–1.0), comfortably above $\tau=0.052$ and, on average, above F^* too, since neither stage targets F^* directly (C3) – Stage 1's unbounded $\arg \max F$ pushes past it.

Cohort: $n = 46$ per config, all feasible in both arms (both left and right panels) – unlike the original cohort,

WDC: a genuinely large, uncurated corpus

50.8M web tables from Common Crawl – tier_10k and tier_100k, tier_1m pending

Unlike santos/santosLarge, WDC ships **no query/datalake split and no curated query list** – query tables are drawn algorithmically, and (unlike santos-family benchmarks) **the set of query tables is not fixed**: any table that doesn't work out can simply be replaced.

Critical constraint: `tier_10k` $\subset$ `tier_100k` exactly (verified directly: all 10,001 `tier_10k` files appear in `tier_100k`) – so a valid scale comparison must hold the *same* query tables fixed across tiers and observe how the surrounding datalake's growth changes outcomes, not optimize a different query set per tier.

The old approach (superseded here) inflated $|D|$ by **relaxing §7.3's retrieval rule itself** ($D_{sem} \cup D_{ovl}$). This section replaces that with santosLarge's maxD methodology instead – §7.3-faithful intersection retrieval throughout, applied to a single shared query set.

Old union-mode report/figures backed up under `experiments/results/_backup_pre_maxd_wdc/`.

WDC: query selection – one shared set, not per-tier

Candidates drawn from `tier_10k` only; `maxD` choice made using `tier_100k`

1. Candidates drawn from `tier_10k`'s table universe only (guaranteed present in every larger nested tier).
2. `maxD` attribute/value selection against the larger `tier_100k` context (rank by free posting-list proxy, verify top 20 by real retrieval, keep the largest $|D|$).
3. That fixed (table,attr,value) triple is evaluated for real $|D|$ against every tier's own index.

`top_n=5000`. `tier_10k` $\subset$ `tier_100k` $\subset$ `tier_1m` nesting verified directly.

WDC: an unclamped-by-construction grid – santosLarge rigor, applied here

Revised after finding the original grid was 87% clamped

An initial 30-cell grid ($k \in \{10..50\} \times \alpha \in \{10..60\}$, max pool 3,000) never required $|D| \geq \max(\alpha k)$ the way santosLarge's cohort did – measured directly, **87.1% of its 900 (query,cell) pairs were already clamped** at top_n=5000. Reverted to santosLarge's discipline instead:

1. Widen the candidate pool to **210** (60 original + 150 more, same seeded shuffle).
2. Use $\min(D_{10k}, D_{100k})$ as the binding threshold; derive the grid from the observed distribution: $k \in \{5, 10, 20\} \times \alpha \in \{5, 10\}$ – 6 cells, pools 25–200.
3. Retain every candidate with $\min(D_{10k}, D_{100k}) \geq 200$ – guaranteed unclamped on *both* tiers, every cell.

67 of 210 retained (31.9%) – comparable in spirit to santosLarge's 47/78.

experiments/wdc/wdc_maxd_selection_unclamped.py; notes in
experiments/results/wdc_unclamped_selection_notes.md.

WDC: choosing F^*, δ, τ – 100% joint feasibility

τ floor 0.05, same round values as before

Swept for *joint* feasibility on the new 6-cell grid + 67-candidate pool: a query only counts if every one of its 6 cells is feasible on *both* tiers simultaneously.

Chosen: $F^*=0.20$, $\delta=0.10$, $\tau=0.10$:

	tier_10k feasible	tier_100k feasible	feasible on BOTH
of 67 candidates	67/67	67/67	67/67 (100%)

Every candidate that cleared the $|D| \geq 200$ threshold is also fully feasible – no outlier exclusion needed, unlike the original design.

WDC: final query cohort – 67 tables, identical across tiers

100% retention at the chosen threshold/grid

All 67 query tables (same attribute, same value) used against every tier – no exclusions.

- ▶ Threshold: $\min(D_{10k}, D_{100k}) \geq 200$, grid $k \in \{5, 10, 20\} \times \alpha \in \{5, 10\}$ (max pool 200).
- ▶ All 67 feasible across all 6 grid cells, on *both* tiers.

Final cohort: `experiments/results/wdc_unclamped_final_cohort.csv`. `tier_1m`'s embeddings/metadata store are built (`tier_100k \subset tier_1m` nesting verified) – full end-to-end study on this 67-query cohort, all three tiers, still pending.

WDC: one-off index cost, all three tiers

HNSW size scales linearly; build time does not scale uniformly

tier	tables	HNSW build	HNSW size	inv. build	inv. size
tier_10k	10,000	13.9 s	163.6 MB	2.7 s	8.75 MB
tier_100k	100,000	200.8 s	1,633.0 MB	23.4 s	76.6 MB
tier_1m	999,999	1,284.8 s	16,322.8 MB	49.6 s	670.5 MB

HNSW size scales almost perfectly linearly throughout (163.6 MB $\rightarrow$ 1,633.0 MB $\rightarrow$ 16,322.8 MB – $\sim 10\times$ at every $10\times$ step) – expected, one fixed-size embedding per table regardless of scale. **HNSW build time scaling is NOT uniform**: $\sim 14.5\times$ from tier_10k $\rightarrow$ tier_100k (superlinear, $O(n \log n)$ -consistent) but only $\sim 6.4\times$ from tier_100k $\rightarrow$ tier_1m (sublinear) – not investigated further here, but a real, reported pattern rather than a single clean exponent. **Inverted-index vocabulary grows sublinearly at every step** (995,957 $\rightarrow$ 6,444,399 distinct values, $\sim 6.5\times$ for $10\times$ tables): common values keep getting reused rather than each new table contributing an entirely new vocabulary.

Sizes are on-disk after native serialization (`hnsplib.save_index`; pickled postings dict) – neither index is persisted by the normal pipeline, both rebuilt in-memory on every `build_wdc_context` call. tier_1m required a memory-frugal rewrite of the measurement script: this account has a hard 32GB per-process RSS limit (`ulimit -m`, confirmed non-adjustable), and the straightforward approach (keeping the raw vectors dict, its categorical-filtered copy, AND hnsplib's own internal copy alive at once) peaked near that ceiling and was twice silently killed. Script: `experiments/wdc/wdc_index_size_report.py`

HNSW size, explained: it's categorical columns, not tables

santosLarge vs. all three WDC tiers – same underlying constant

HnswRetriever only ever indexes **categorical columns** ($\theta_{\text{cat}}=50$ -bounded):

`categorical_only_vectors` zeroes every non-categorical column's embedding before it reaches `hnswlib`, and `_flatten` skips zero-norm rows – so the indexed population matches the overlap index's exactly.

dataset	total cols	cat. cols (indexed)	HNSW size	bytes/elem
santosLarge	121,796	84,157 (69.1%)	281.78 MB	3,511.4
tier_10k	51,854	48,857 (94.2%)	163.59 MB	3,512.0
tier_100k	518,131	487,728 (94.1%)	1,633.02 MB	3,512.6
tier_1m	5,177,216	4,874,987 (94.2%)	16,322.76 MB	3,512.1

Bytes/element is a true constant, $\sim 3,512$, across all four datasets (768-dim float32 vector + `hnswlib` graph links at $M=32$) – so index size is exactly (indexed elements) $\times$ (fixed per-element cost), nothing else. `santosLarge`'s index is $\sim 58\times$ smaller than `tier_1m`'s purely because it has $\sim 58\times$ fewer categorical columns (84,157 vs. 4,874,987) – not because its tables are smaller (they aren't: `santosLarge`'s are far *bigger*, $\sim 5,000$ rows vs. WDC's ~ 6). **Coverage ratio explains why:** `santosLarge`'s larger tables make $\theta_{\text{cat}}=50$ a real, selective filter (69.1%), while WDC's tiny tables (median ~ 6 rows) mean `nunique(col)` is capped by row count itself, so almost every column qualifies as categorical (94.1–94.2%, flat across all three tiers).

Inverted index size, explained: two different growth curves

Not a fixed per-element cost like HNSW – postings and vocabulary scale differently

dataset	cat. cols	distinct vals	postings	avg dom/col	avg fan-out
santosLarge	84,157	277,704	887,428	10.54	3.20
tier_10k	48,857	141,737	329,064	6.74	2.32
tier_100k	487,728	995,957	3,347,443	6.86	3.36
tier_1m	4,874,987	6,444,399	33,453,833	6.86	5.19

Total postings tracks categorical-column count almost exactly linearly (avg domain size/column is flat per dataset: ~ 10.5 for santosLarge, ~ 6.7 – 6.9 for WDC, capped by WDC's ~ 6 -row median table size, not corpus scale) – this is why postings grow $\sim 10\times$ per $10\times$ WDC tables, matching column-count growth. **Vocabulary (distinct values) grows sub-linearly instead**: common web-table values (“1”, “yes”, country names, ...) keep getting reused rather than each new table contributing new ones, so **average fan-out climbs with scale** ($2.32 \rightarrow 3.36 \rightarrow 5.19$) – the same value attaches to more (table,attr) pairs as WDC grows.

Size itself is a **pickle-framing proxy, not a native format** (unlike HNSW's `save_index`) – bytes/posting stays roughly stable (~ 20 – 28 , pickle memoizes repeated table-name strings) but **bytes/distinct-value rises with scale** ($63.7 \rightarrow 64.7 \rightarrow 80.6 \rightarrow 109.1$) since each key's Set payload grows with fan-out. Net effect: tier_1m's inverted index is $\sim 40\times$ larger than santosLarge's (670.45 MB vs. 16.86 MB) – smaller than the $\sim 58\times$ column-count gap, because WDC's lower avg domain/column partly offsets its far larger column count.

avg dom/col = postings/cat. cols; avg fan-out = postings/distinct values. Same source data as the previous slide's inverted-index sizes.

WDC: Dinkelbach convergence vs. input pool size, all three tiers

$k=20$, $\alpha=10$ (pool= 200, the grid's own ceiling) – 67-query cohort, all 6 top_n values

tier	top_n	mean $ D $	n_ran	n_clamped	mean stage-1	mean iters
tier_10k	5,000	387.0	67	0	0.67 ms	2.16
tier_10k	10,000	618.4	67	0	1.26 ms	2.48
tier_10k	15,000	776.2	67	0	1.53 ms	2.48
tier_10k	20,000	854.1	67	0	1.72 ms	2.57
tier_10k	25,000	877.9	67	0	1.80 ms	2.57
tier_10k	30,000	882.6	67	0	1.88 ms	2.58
tier_100k	5,000	925.5	67	0	1.45 ms	1.99
tier_100k	10,000	1,409.1	67	0	2.25 ms	2.22
tier_100k	15,000	1,755.3	67	0	3.30 ms	2.31
tier_100k	20,000	2,048.0	67	0	4.08 ms	2.37
tier_100k	25,000	2,371.9	67	0	5.33 ms	2.42
tier_100k	30,000	2,630.7	67	0	6.44 ms	2.51
tier_1m	5,000	1,338.7	62	5	2.17 ms	1.58
tier_1m	10,000	2,629.7	66	1	4.82 ms	1.77
tier_1m	15,000	3,440.9	65	2	6.77 ms	1.85
tier_1m	20,000	4,215.8	66	1	7.87 ms	1.88
tier_1m	25,000	4,971.4	65	2	11.04 ms	1.86
tier_1m	30,000	5,633.9	65	2	11.56 ms	1.97

Iteration count stays essentially flat (1.58–2.58) across three orders of magnitude of corpus size, direct empirical support for the paper's “converges in only a few iterations” claim (§5). Stage-1 time tracks $|D|$, not $|T|$ and stays under 12 ms everywhere. Clamping is essentially absent at this grid's own ceiling: 0/67 on tier_10k/tier_100k, only 1–5/67 on tier_1m (mild non-monotonicity, not a bug).

tier_1m's context required a batched-add_items HNSW build plus $M=16$ (vs. $M=32$ elsewhere) to fit this account's 32GB RSS ceiling – a memory-driven exception, not a scalability finding; index now cached to disk. Script: experiments/wdc/wdc_topn_pool_sweep.py.

WDC: Dinkelbach convergence at a second grid cell – $k=10, \alpha=3$ (pool= 30)

Same 67-query cohort, same top_n sweep – a smaller, easier pool

tier	top_n	mean $ D $	n_ran	n_clamped	mean stage-1	mean iters
tier_10k	5,000	387.0	67	0	0.54 ms	2.06
tier_10k	10,000	618.3	67	0	0.87 ms	1.94
tier_10k	15,000	775.7	67	0	1.11 ms	2.00
tier_10k	20,000	853.3	67	0	1.21 ms	1.93
tier_10k	25,000	877.2	67	0	1.23 ms	1.93
tier_10k	30,000	881.9	67	0	1.27 ms	1.93
tier_100k	5,000	807.9	67	0	1.31 ms	1.76
tier_100k	10,000	1,289.4	67	0	1.90 ms	1.75
tier_100k	15,000	1,571.3	67	0	2.12 ms	1.66
tier_100k	20,000	1,847.5	67	0	2.47 ms	1.73
tier_100k	25,000	2,111.3	67	0	3.63 ms	1.75
tier_100k	30,000	2,368.6	67	0	3.54 ms	1.70
tier_1m	5,000	1,071.4	66	1	1.42 ms	1.47
tier_1m	10,000	1,898.0	66	1	2.81 ms	1.48
tier_1m	15,000	2,840.6	66	1	4.90 ms	1.61
tier_1m	20,000	3,559.3	66	1	6.13 ms	1.52
tier_1m	25,000	4,355.4	66	1	7.61 ms	1.52
tier_1m	30,000	5,133.4	66	1	10.57 ms	1.61

Same qualitative findings, more strongly: mean iterations are lower still (1.47–2.06 vs. 1.58–2.58) – a smaller pool is a cheaper top-k selection. **Clamping is even rarer:** 0/67 on tier_10k/tier_100k, a consistent 1/67 on tier_1m at every top_n.

tier_1m loaded its index from the previous slide's disk cache (seconds, not ~14 min). Caveat: tier_100k's mean $|D|$ differs slightly from the previous slide's despite retrieval not depending on k/α – hnsplib's multi-threaded build is seed-deterministic in its own choices but not concurrent insertion order, so independent rebuilds over identical data can differ slightly. Full data: wdc_topn_pool_sweep_k10_a3.csv.

WDC: Stage 2 execution time – LP pre-check vs. MILP solver, all three tiers

top_n= 5000, $\alpha \in \{10, 5\} \times k \in \{5, 10, 20\}$, 67-query cohort – WDC analogue of santosLarge \$5c

tier	α	k	pool	clamped	feasible	LP precheck	MILP solver	pre+solver	full wall
tier_10k	10	5	50	0	67/67	1.071 ms	1.583 ms	2.654 ms	2.905 ms
tier_10k	10	10	100	0	67/67	0.994 ms	1.697 ms	2.691 ms	2.938 ms
tier_10k	10	20	200	0	67/67	1.202 ms	2.162 ms	3.364 ms	3.651 ms
tier_10k	5	5	25	0	67/67	0.873 ms	1.426 ms	2.299 ms	2.530 ms
tier_10k	5	10	50	0	67/67	0.891 ms	1.467 ms	2.359 ms	2.593 ms
tier_10k	5	20	100	0	67/67	0.980 ms	1.677 ms	2.658 ms	2.905 ms
tier_100k	10	5	50	0	67/67	1.183 ms	2.148 ms	3.331 ms	3.625 ms
tier_100k	10	10	100	0	67/67	1.181 ms	2.887 ms	4.068 ms	4.364 ms
tier_100k	10	20	200	0	67/67	1.376 ms	4.341 ms	5.717 ms	6.048 ms
tier_100k	5	5	25	0	67/67	0.992 ms	2.322 ms	3.314 ms	3.583 ms
tier_100k	5	10	50	0	67/67	1.019 ms	2.122 ms	3.141 ms	3.421 ms
tier_100k	5	20	100	0	67/67	1.135 ms	3.221 ms	4.357 ms	4.648 ms
tier_1m	10	5	50	1	60/67	1.682 ms	2.153 ms	3.835 ms	3.912 ms
tier_1m	10	10	100	1	60/67	1.201 ms	2.210 ms	3.411 ms	3.449 ms
tier_1m	10	20	200	4	60/67	1.354 ms	2.705 ms	4.059 ms	4.073 ms
tier_1m	5	5	25	1	60/67	0.976 ms	1.706 ms	2.682 ms	2.746 ms
tier_1m	5	10	50	1	60/67	0.990 ms	1.812 ms	2.802 ms	2.860 ms
tier_1m	5	20	100	1	60/67	1.086 ms	1.962 ms	3.048 ms	3.098 ms

Stage 2 is a few ms everywhere (2.3–6.0 ms), growing ~ 1.5 – $1.7\times$ for an $8\times$ larger pool and only $\sim 1.4\times$ for $100\times$ more tables; the MILP is ~ 55 – 76% of pre+solver. **New at tier_1m: the LP pre-check becomes decisive** – 7/67 queries are LP-infeasible at every cell (MILP skipped), since the cohort's τ was validated only on the two smaller tiers.

U is seeded synthetic uniform(0,1), not real unionability (tier_1m's lean context has no full vectors dict); retrieval re-run per tier, so $|D|$ is not bit-identical to earlier slides for the smaller tiers; tier_1m uses the cached $M=16$ index. LP averaged over all rows, MILP over LP-feasible rows only. Script: wdc_stage2_timing.py.

WDC tier_1m: what the LP pre-check saves on the 7 infeasible queries

MILP skipped when the LP proves infeasibility – measured by re-running those 7 pools with and without the pre-check

α	k	pool	with pre-check	MILP only (no pre-check)	saved / query
10	20	200	1.004 ms	1.102 ms	0.098 ms
10	10	100	0.862 ms	1.096 ms	0.234 ms
10	5	50	0.895 ms	1.133 ms	0.238 ms
5	20	100	0.839 ms	1.033 ms	0.194 ms
5	10	50	0.763 ms	1.005 ms	0.242 ms
5	5	25	0.738 ms	1.015 ms	0.277 ms

The saving is small: ~ 0.1 – 0.28 ms per skipped query, ~ 0.7 – 1.9 ms across all 7. HiGHS also detects infeasibility quickly on its own, so skipping the MILP avoids ~ 1 ms of work while the LP itself costs ~ 0.7 – 0.9 ms – at these pool sizes the pre-check is close to break-even. Its value here is soundness and avoiding branch-and-bound on harder instances, which these pools are not.

Median of 5 in-process repetitions per pool, warm caches, so absolute values are lower than the previous slide's means (which include cold runs and feasible rows); treat as $\pm 30\%$. Both variants returned infeasible on every pool. Same seeded synthetic U and $\tau=0.10$ as the previous slide.

WDC: per-stage and end-to-end time, real unionability, all three tiers

top_n= 5000, $\alpha \in \{10, 5\} \times k \in \{5, 10, 20\}$, 67-query cohort – mean ms

tier	α	k	pool	clamp	feas.	retrieval	Stage 1	unionab.	LP	MILP	Stage 2	end-to-end
tier_10k	10	5	50	0	67/67	15.9	0.62	12.83	1.57	2.23	4.05	33.4
tier_10k	10	10	100	0	67/67	15.9	0.89	24.96	1.27	2.22	3.73	45.4
tier_10k	10	20	200	0	67/67	15.9	0.98	45.94	1.53	2.81	4.62	67.4
tier_10k	5	5	25	0	67/67	15.9	0.93	8.28	1.02	1.85	3.08	28.2
tier_10k	5	10	50	0	67/67	15.9	0.79	13.85	1.05	1.98	3.25	33.7
tier_10k	5	20	100	0	67/67	15.9	0.88	25.06	1.19	2.21	3.64	45.4
tier_100k	10	5	50	0	67/67	33.0	1.19	12.55	1.57	2.26	4.09	50.8
tier_100k	10	10	100	0	67/67	33.0	1.76	26.69	1.28	2.43	3.96	65.4
tier_100k	10	20	200	0	67/67	33.0	1.85	49.08	1.44	2.96	4.67	88.6
tier_100k	5	5	25	0	67/67	33.0	1.75	7.66	0.99	1.85	3.05	45.5
tier_100k	5	10	50	0	67/67	33.0	1.61	13.14	0.98	1.88	3.08	50.8
tier_100k	5	20	100	0	67/67	33.0	1.70	25.35	1.18	2.28	3.70	63.8
tier_1m	10	5	50	1	60/67	351.3	1.46	11.71	2.14	3.42	5.48	370.0
tier_1m	10	10	100	1	60/67	351.3	2.07	24.52	1.27	2.37	3.62	381.5
tier_1m	10	20	200	4	60/67	351.3	2.07	46.01	1.41	5.40	6.49	405.9
tier_1m	5	5	25	1	60/67	351.3	1.99	7.03	1.01	1.86	2.86	363.2
tier_1m	5	10	50	1	60/67	351.3	1.93	12.68	1.01	1.94	2.95	368.9
tier_1m	5	20	100	1	60/67	351.3	1.95	24.12	1.15	2.58	3.66	381.1

Retrieval is the only stage that grows with the corpus: 15.9 $\rightarrow$ 33.0 $\rightarrow$ 351.3 ms per query, 86–97% of end-to-end on tier_1m. **Unionability scoring is linear in the pool, independent of the corpus:** ~ 0.23 – 0.26 ms per pool table on every tier (200 tables: 45.9 / 49.1 / 46.0 ms) – the αk efficiency claim, measured. Stage 1 (0.6–2.1 ms) and Stage 2 (2.9–6.5 ms) stay small and scale-free.

Real pinned-match U ($\sigma=0.6$), supersedes the synthetic- U Stage 2 slide. Retrieval (HNSW probe + overlap filter + N_i/n_i lookup) is independent of k, α and timed once per query. tier_1m: cached $M=16$ index, peak RSS ≈ 31.8 GiB of the 32 GiB cap (tight). MILP averaged over LP-feasible rows. Details: wdc-report.md §5.

WDC: why tier_1m retrieval is slow – HNSW vs. overlap probe

top_n= 5000, 67-query cohort, mean per query; retrieval = HNSW probe + overlap probe + join + N_i/n_i lookup

tier	HNSW probe (ms)	overlap probe (ms)	join (ms)	lookup (ms)	total (ms)	$ D_{ovl} $	$ D $
tier_10k	8.1	1.2	1.0	1.2	11.4	3,289	388
tier_100k	9.3	15.5	2.1	2.9	29.9	34,038	888
tier_1m	10.2	203.5	24.4	3.6	241.7	341,014	1,071

The HNSW probe is almost flat (8.1 → 9.3 → 10.2 ms for 100× more tables). **The overlap probe is the cause:** it returns every (table, column) pair containing the protected value, and that set grows $\sim 10\times$ per tier (3.3k → 34k → 341k), so its time grows $\sim 13\times$ then $\sim 13\times$ (1.2 → 15.5 → 203.5 ms) – 84% of tier_1m retrieval. The join (dict + set intersection over those pairs) grows the same way (24.4 ms at 1M). Only $\sim 1,000$ pairs survive the intersection, so almost all of the overlap work is thrown away.

Same query grid and cached indexes as the previous slide; repeating each probe immediately gives nearly the same time (HNSW 8.6 ms, overlap 196 ms at 1M), so this is not cold-cache or paging cost. Totals (241.7 ms) are lower than the 351 ms above because this run had no 32 GiB memory cap (peak 46 GiB), so the two are different machine conditions; the split is the finding. Details: wdc-report.md §5.1.

Benefit of Stage 1: post-retrieval pipeline time

top_n= 30000, $k=10$, $\alpha=3$ (pool $\alpha k=30$) – mean ms per query, timed from just before Stage 1 to the end of the pipeline

Dataset	n	$ D $	Two-stage (scores $\alpha k=30$)				Stage 1 skipped (scores $ D $)			Saving	
			S1	score	S2	total	score	S2	total	%	×
Santos-Large	46	259	0.29	39.68	3.18	43.2	386.5	4.0	390.5	88.9	9.0
WDC-10K	67	881	1.33	9.21	2.10	12.6	227.9	10.5	238.4	94.7	18.9
WDC-100K	67	2518	3.49	7.06	2.84	13.4	433.0	36.9	470.0	97.1	35.1
WDC-1M	66	5211	8.82	6.46	4.85	20.1	667.3	525.8	1193.1	98.3	59.3

Same retrieved D in both conditions, so they differ only in whether Stage 1 runs. **Two-stage** = Stage 1 (Dinkelbach) → score U on the 30-table pool → Stage 2 over the pool. **Stage 1 skipped** = score U on all of D → Stage 2 over D at cardinality k . Saving = $(T_{\text{skip}} - T_{\text{two}})/T_{\text{skip}}$.

Retrieval, the overlap intersection and the N_i/n_i lookup are outside the timed window (identical in both). F^*/δ : 0.122/0.07 on Santos-Large, 0.20/0.10 on the WDC tiers. HNSW indexes are cached to disk, so $|D|$ is reproducible across runs. One WDC-1M query has $|D|=2 < k$ and is excluded. Details: stage1-benefit-report.md.

What the Stage 1 table shows

In plain terms

- ▶ **The bigger the data lake, the more Stage 1 saves:** 88.9% on Santos-Large, 94.7% on WDC-10K, 97.1% on WDC-100K, 98.3% on WDC-1M (9.0 $\times$ to 59.3 $\times$). Stage 1 caps unionability computations at $\alpha k = 30$ no matter how many candidates retrieval returns, so the larger $|D|$ grows, the more work it removes.
- ▶ **Almost all of the saving is unionability scoring** – 386.5 ms $\rightarrow$ 39.7 ms on Santos-Large, 667.3 ms $\rightarrow$ 6.5 ms on WDC-1M. That is 30 bipartite matchings instead of $|D|$ of them: the αk -not- $|\mathcal{T}|$ claim, measured.
- ▶ **At WDC-1M, skipping Stage 1 also makes Stage 2 expensive:** its ILP runs over $\sim 5,211$ candidates instead of 30, costing 525.8 ms against 4.8 ms. Both the scoring and the solving inflate, not just the scoring.
- ▶ **Stage 1 is nearly free:** 0.29–8.8 ms, against savings of hundreds of ms.
- ▶ **Feasibility is unaffected** – 46/46, 67/67, 67/67, 64/66, identical in both conditions. The distribution-aware pre-filter never cost a feasible answer here.
- ▶ **But the speedup is not free in quality:** the pool reaches 80–91% of the ΣU that scoring every candidate achieves. Stage 1 can exclude a high- U table that Stage 2 would otherwise have chosen.

Absolute ms vary with machine load between runs (the same cached $|D|$ gave 181.8 and 238.9 ms for WDC-10K's skip condition); the *ratio* is the stable quantity. $|D|$ is right-skewed on WDC-1M (mean 5,211, median $\approx 3,557$, max 16,715), so the median per-query saving there is 97.7% against a mean of 98.3%. $top_n=30000$ is the deepest retrieval setting and so the largest $|D|$; the saving narrows at smaller top_n .

Takeaways

- ▶ The pipeline has **four** short-circuit points – two before Stage 1 even runs (`insufficient_candidates`, the C5 clamp), two around Stage 2 (LP-decise, `stage2_infeasible`).
- ▶ **santosLarge** (redesigned: attributes/values deliberately chosen to maximize $|D|$, $|D| \geq 60$ cohort, $F^*=0.122/\delta=0.07/\tau=0.052$ chosen to maximize reach without a degenerate τ): the pipeline shows scoring cost scaling with αk , Stage 1 buying a real 1.7–2.8 $\times$ speedup at a 5–9% ΣU cost, and (unlike the original random-assignment cohort, where the LP pre-check was decisive on 37% of instances) every one of the 46 retained queries is feasible in all 6 configs, so the LP pre-check has nothing left to catch here either.
- ▶ **WDC, all three tiers** (redesigned, replacing the old $D_{sem} \cup D_{ovl}$ relaxation): santosLarge's maxD attribute/value selection, applied under §7.3-faithful intersection retrieval to **the same query tables at every scale** ($\text{tier}_{10k} \subset \text{tier}_{100k} \subset \text{tier}_{1m}$, verified) – not independently-optimized per-tier cohorts. An initial 30-cell grid proved 87.1% clamped; reverted to a santosLarge-style unclamped-by-construction 6-cell grid ($k \in \{5, 10, 20\} \times \alpha \in \{5, 10\}$, max pool 200) over a widened 210-candidate pool, with $F^*=0.20/\delta=0.10/\tau=0.10$ (τ floor 0.05) achieving **100% joint feasibility (67/67)** on both tiers simultaneously. At $k=20, \alpha=10$ (pool=200), Dinkelbach converges in 1.58–2.58 mean iterations and under 12 ms mean Stage-1 time **across all three tiers, three orders of magnitude of table count** – clamping is essentially absent (0/67 on `tier_10k/tier_100k`, 1–5/67 on `tier_1m`, a mild non-monotonicity effect, not a bug).
- ▶ **Together**: santosLarge answers "does the two-stage design's efficiency claim hold when it has room to matter?"; WDC (all three tiers, redesigned cohort) confirms it does on an uncurated corpus too, at genuine million-table scale.